

If you haven't yet committed the changes, then:

```
git diff > mypatch.patch
```

But sometimes it happens that part of the stuff you're doing are new files that are untracked and won't be in your git diff output. So, one way to do a patch is to stage everything for a new commit (git add each file, or just git add .) but don't do the commit, and then:

```
git diff --cached > mypatch.patch
```

Add the 'binary' option if you want to add binary files to the patch (e.g. mp3 files):

```
git diff --cached --binary > mypatch.patch
```

You can later apply the patch:

```
git apply mypatch.patch
```

ShareImprove this answerFollow

edited Nov 6, 2020 at 5:52

answered Mar 15, 2013 at 17:43



jcarballo

28.8k 3 29 29

-
- 7 I did exactly that and got "fatal: unrecognized input" upon executing git apply. Any idea what can cause this and how to fix it? – Vitaly Dec 22, 2013 at 20:11
- 11 @Vitaly: is your patch readable if you open it with a text editor? it should be clean with no strange characters, for example if the color.diff setting is set your patch will have some 'color characters' that can make 'git apply' fail, in that case try git diff --no-color . Otherwise, it looks like an encoding problem. – jcarballo Dec 22, 2013 at 21:06
- 12 To create the patch from the already staged changes you could also do git diff --staged > mypatch.patch , because --staged is a synonym for --cached . I think it easier to remember. – matthaeus Mar 3, 2017 at 16:52
- 4 Related to "new files that are untracked": "git diff" and "git diff --cached" only work if "git add <file>" has been called first. (I am new to git and wondered why I got an empty patch everytime) – Anonymous Apr 25, 2017 at 8:41
- 5 This got me out of a strange merge/rebase hell pretty easily, thanks :) – John Hunt Aug 31, 2017 at 10:34

Show 8 more comments

Report this ad



git diff for unstaged changes.

620

git diff --cached for staged changes.



git diff HEAD for both staged and unstaged changes.



ShareImprove this answerFollow

edited Sep 5, 2020 at 19:30

answered Mar 1, 2011 at 19:16



sigjuice

29.6k

14

70

95



-
- 21 yup, git diff is the inverse of git apply – Spike Gronim Mar 1, 2011 at 19:20
- 43 git format-patch also includes binary diffs and some meta info. Actually that would be the best bet for creating a patch, but afaik this does only work for checked in sources/ changes, right? – Eric Mar 18, 2012 at 12:24
- 27 Sometimes it might be useful to create a patch relative to the current directory. To achieve this, use git diff --relative – ejboy Jan 8, 2013 at 14:03
- 38 git diff > a.patch to write it to a file – qasimzee Feb 12, 2013 at 11:33
- 155 Terse bordering on sarcastic, the answer below is more helpful. – Air Dec 7, 2013 at 19:03

Show **8** more comments